

README

Browser-runtime proof — vanilla qBittorrent WebUI + Angular dashboard

Revision: 1 Last modified: 2026-09-01T16:20:00Z

What this run proves

The claim “the vanilla qBittorrent WebUI works through the proxy” previously rested only on HTML-integrity assertions (namespace token present, zero rebrand). Those bytes look identical whether the JavaScript runs or is dead — which is exactly how the removed themed overlay slipped through: it rewrote qBittorrent → Ёo6a inside inline `<script>` blocks, destroying `window.qBittorrent`, while the served HTML still passed every byte check.

This run replaces that with **runtime** evidence from a real browser.

Environment

Item	Value
Browser	Chromium 152.0.7977.64 (snap), headless
Node	v26.8.1
Playwright	@playwright/test ^1.59.1
Config	frontend/ playwright.runtime.config.ts
Targets	

Item	Value
	http://localhost:7186 (proxy, user-facing) · http://localhost:7187 (Angular)
Host safety	nice -n 19 ionice -c 3, headless, 1 worker, 0 retries

Playwright's bundled Chromium cannot be installed on this host — playwright install chromium fails with ERROR: Playwright does not support chromium on ubuntu26.04-x64. The config therefore drives the host's own Chromium via executablePath (auto-detected; override with BOBA_CHROMIUM_PATH).

Result — 5/5 passed, 3/3 consecutive runs identical

- ✓ Angular dashboard (:7187) hydrates <app-root> with zero console errors
- ✓ negative control: a wrong password does NOT log in
- ✓ real UI login executes the WebUI JS with zero console errors
- ✓ MUTATION: rebranding "qBittorrent" inside client.js breaks the namespace guard
- ✓ MUTATION: blocking main bundle leaves the Angular <app-root> shell empty

Verbatim output: [playwright-run.log](#)

Console-error inventory

Surface	Uncaught exceptions	Console errors (excl. control needle)
qBittorrent WebUI, load → login → logged in (:7186)	0	0
Angular dashboard (:7187)	0	0

Non-error observations, classified:

Observation	Classification	Note
GET http://localhost:7186/ → net::ERR_ABORTED POST .../api/v2/auth/login → net::ERR_ABORTED	pre-existing in vanilla qBittorrent	scripts/login.js runs location.replace(location); location.reload(true); on a successful login, which cancels the in-flight navigation. Not a console error, not an exception, not proxy-caused.
401 Unauthorized console error in the negative control	expected / by design	The wrong-password rejection. Present only in that test, which does not assert zero.

Anti-bluff measures (why the zeros are trustworthy)

1. **Control needle (§11.4.201(6)(7)(b)).** A blind listener and a clean page both report zero. Each spec injects a known console.error on the page under test and asserts the listener *observed* it before trusting silence. Recorded as controlNeedleObserved: true in the evidence JSON.
2. **Negative control.** A wrong password must not log in. It also fails loudly if the refusal came from an IP **ban** rather than credential rejection — a ban would leave the page on the login form and pass for the wrong reason.
3. **Paired §1.1 mutations** (runtime-guards-mutation.spec.ts) reproduce the real historical defect in-browser and prove the guards can fail:
 - rewriting qBittorrent → Ёo6a inside client.js makes Client, Filters, Search, TransferList vanish from the namespace — while the DOM still renders, which is precisely why an HTML check cannot see it;
 - blocking main-*.js leaves <app-root> with 0 children while HTTP still returns 200.
4. **Determinism (§11.4.50).** 3/3 consecutive runs, identical results.

Two defects found in the tests themselves (fixed, recorded)

Both were **test artifacts**, not product defects — recorded because a guard that refuses a healthy page is a §11.4.201(1) false positive:

- **Log wrongly required.** The first namespace list was built from a grep that matched *references*, not *assignments*. No eagerly-loaded script assigns `window.qBittorrent.Log`; it is registered lazily when the Log tab opens. Requiring it failed a healthy page. Removed.
- **First `waitForFunction` fired far too early.** `typeof qBittorrent === 'object'` resolves the instant `monkeypatch.js` creates the object, while `client.js / dynamicTable.js / misc.js` have not run. Sampled there the namespace held only 5 early keys and a `keys.length > 0` check passed — it would have passed with `client.js` completely broken. Now waits for full load and asserts 17 specific members owned by 9 different scripts.
- **Mutation over-claimed.** Statistics was listed as `client.js`-owned but is assigned by `statistics.js`, so it legitimately survives the mutation. Listing it made the fixture report a toothless guard when the guard was fine.

Evidence files

File	Contents
<code>playwright-run.log</code>	Verbatim run output
<code>positive-login-console.json</code>	27 namespace members, 17 Client functions, console/page-error inventory, control needle
<code>negative-control-wrong-password.json</code>	Rejection message, proof <code>window.qBittorrent</code> absent pre-auth
<code>angular-dashboard-console.json</code>	Hydration counts (5 children, 416 chars), zero errors
<code>qbittorrent-webui-logged-in.png</code>	Logged-in WebUI, 1280×720
<code>angular-dashboard-hydrated.png</code>	Hydrated dashboard, 1280×918

qbittorrent-webui-logged-in.png shows the real qBittorrent chrome (File/Edit/View/Tools/Help, torrent table, filter sidebar) with a **live** status bar — Free space: 1.533 TiB, DHT: 122 nodes — i.e. the JS sync loop is actively polling the API, not merely parsed. No 506a rebrand anywhere in the WebUI.

angular-dashboard-hydrated.png shows the hydrated dashboard with live state (qBit Connected – admin, 43 Trackers, tracker chips).

Reproduce

```
cd frontend
nice -n 19 ionice -c 3 ./node_modules/.bin/playwright test \
  --config=playwright.runtime.config.ts
```

Requires the stack live on :7186 and :7187. No container is started, stopped, or restarted by this suite; the mutations rewrite responses **in the browser only** (page.route), never on the server or on disk.

Honest boundary (§11.4.6)

This proves the WebUI's JavaScript **executes**, the namespace is **fully populated**, a real end-user login **works**, and both surfaces are **free of console errors and uncaught exceptions** on Chromium 152 at this revision.

It does **not** prove: every WebUI feature works (only load + login + live sync are driven); behaviour on other browser engines (Firefox/WebKit untested); or that no defect exists outside the exercised paths (§11.4.118 discovery-pressure still applies). WebUI Bridge (down) on the dashboard reflects port 7188, a manually-started host process out of scope here — it is not an error surfaced by this run.